

Approximation Polynomiale et Optimisation Resume

Approximation au sens des moindres carrés

Introduction

- L'interpolation polynomiale peut devenir **instable** hors des points de contrôle (**phénomène de Runge**)
- Le **coût de calcul** augmente fortement avec le nombre de points de contrôle (**nuage de points**)

Régression Linéaire (Méthode matricielle)

$$\underbrace{\begin{pmatrix} e_0 \\ \vdots \\ e_i \\ \vdots \\ e_n \end{pmatrix}}_{\varepsilon} = \underbrace{\begin{pmatrix} 1 & x_0 \\ \vdots & \vdots \\ 1 & x_i \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}}_A \underbrace{\begin{pmatrix} \lambda_0 \\ \lambda_1 \end{pmatrix}}_{\Lambda} - \underbrace{\begin{pmatrix} y_0 \\ \vdots \\ y_i \\ \vdots \\ y_n \end{pmatrix}}_Y$$

$$F(\Lambda) = \|\varepsilon\|_2^2 = e_0^2 + \dots + e_n^2$$

Le vecteur Λ minimisant F vérifie:

$$\nabla F(\Lambda) = \begin{pmatrix} \frac{\partial F}{\partial \lambda_0}(\Lambda) \\ \frac{\partial F}{\partial \lambda_1}(\Lambda) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

(λ_0, λ_1) est un point critique de F

En calculant ∇F , on trouve:

$$\nabla F(\Lambda) = 2 {}^t A(A\Lambda - Y)$$

$$\Rightarrow {}^t A(A\Lambda - Y) = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \Rightarrow {}^t A A \Lambda = {}^t A Y$$

D'où:

$\Lambda = ({}^t A A)^{-1} {}^t A Y$

On peut aussi montrer que:

$$\Lambda = \begin{pmatrix} \lambda_0 \\ \lambda_1 \end{pmatrix} = \begin{pmatrix} \overline{Y - \lambda_1 \overline{X}} \\ \frac{\overline{XY} - \overline{X} \overline{Y}}{\overline{X^2} - (\overline{X})^2} \end{pmatrix}$$

avec:

$$\overline{X} = \frac{1}{n+1} \sum_{i=0}^n x_i, \quad \overline{Y} = \frac{1}{n+1} \sum_{i=0}^n y_i, \quad \overline{XY} = \frac{1}{n+1} \sum_{i=0}^n x_i y_i, \quad \overline{X^2} = \frac{1}{n+1} \sum_{i=0}^n x_i^2$$

```
1 import numpy as np
2 from typing import Tuple
3
4 def matrice(X: np.ndarray, p: int) -> np.ndarray:
5     n, A = len(X), np.zeros((n, p + 1))
6     for j in range(p + 1):
7         A[:, j] = X[:, 0] ** j
8     return A
```

```

9
10 def regression(X: np.ndarray, Y: np.ndarray, p: int) -> np.ndarray:
11     A = matrice(X, p)
12     return np.linalg.inv(A.T @ A) @ A.T @ Y[:, 0]

```

Inconvénients de la résolution analytique

Instabilité numérique La matrice tAA peut être mal conditionnée (i.e. de petites perturbations des données entraînent de grandes erreurs sur la solution) ou non inversible en cas de multicollinéarité (i.e. lorsque certaines variables sont linéairement dépendantes).

Inadaptée aux grandes bases de données La méthode nécessite de charger l'ensemble des données en mémoire, ce qui devient difficile pour de très grands ensembles (en particulier lorsque le nombre d'observations ou le nombre de variables est très élevé).

Pas de mise à jour incrémentale L'ajout de nouvelles données impose un recalcul complet de $(^tAA)^{-1}$, ce qui n'est pas optimal lorsque les données arrivent en flux continu.

Coût de calcul élevé Le calcul de $(^tAA)^{-1}$ a une complexité de $\mathcal{O}(p^3)$ (où p est le nombre de paramètres ou de variables). Cette opération devient très coûteuse pour des modèles de grande dimension ou des polynômes de haut degré.

Alternative

Utiliser une méthode numérique comme **la descente de gradient**, qui contourne ces limitations.

Méthode de descente de gradient

On considère un ensemble de données:

$$\{(x_i, y_i)\}_{i=1}^n$$

et un modèle linéaire:

$$h_{\theta}(x_i) = \theta_1 x_i + \theta_0$$

Fonction coût

On définit d'abord la **SSE (Sum of Squared Errors)**:

$$\text{SSE}(\theta_0, \theta_1) = \sum_{i=1}^n (h_{\theta}(x_i) - y_i)^2$$

Puis, la **MSE (Mean Squared Error)**:

$$\text{MSE}(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x_i) - y_i)^2$$

Enfin, la fonction coût (**Cost/Loss function**) utilisée pour la descente de gradient :

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x_i) - y_i)^2$$

- SSE mesure l'erreur totale
- MSE est la moyenne de cette erreur
- Le facteur $\frac{1}{2}$ dans J est ajouté pour simplifier les dérivées lors de l'optimisation

$$J = \frac{1}{2} \times \text{MSE} = \frac{1}{2n} \times \text{SSE}$$

Règle de mise à jour (descente de gradient)

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

où $\alpha > 0$ est le taux d'apprentissage.

Taux d'apprentissage α (learning rate)

- α contrôle la taille du pas lors de la mise à jour des paramètres
- C'est un hyperparamètre, il n'est pas appris par le modèle
- Il n'existe pas de méthode exacte pour trouver le meilleur α
- En pratique, le choix de α se fait généralement par essais et erreurs (**tuning**)

Effet du choix de α

- α trop petit: convergence lente
- α trop grand: divergence ou oscillations
- α bien choisi: convergence stable et rapide

Valeurs usuelles

$$\alpha \in \{0.1, 0.01, 0.001, 0.0001\}$$

► Des méthodes comme **Adagrad**, **RMSProp** et **Adam** permettent d'adapter automatiquement le taux d'apprentissage pendant l'entraînement.

Forme vectorielle générale

Soit:

$$\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \end{pmatrix}, \quad X = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

Alors:

$$J(\theta) = \frac{1}{2n} \|X\theta - Y\|^2$$

et le gradient:

$$\nabla J(\theta) = \frac{1}{n} X^T (X\theta - Y)$$

```
1 import numpy as np
2 from typing import Tuple, List
3
4 def gradient_descent(X, Y, p, Lambda0, eta, epsilon, max_iter)
5     -> Tuple[np.ndarray, List[float], List[np.ndarray]]:
6     A = matrice(X, p)
7     Lambda = Lambda0.copy()
8     J_hist, Grad_hist = [], []
9     Y = Y[:, 0]
10    for _ in range(max_iter):
11        error = A @ Lambda - Y                # Gradient
12        gradient = 2 * (A.T @ error)
13        Grad_hist.append(gradient)
14        Lambda = Lambda - eta * gradient        # Update
```

```

15         J = np.linalg.norm(error) ** 2           # Cost function
16         J_hist.append(J)
17         if np.linalg.norm(gradient) < epsilon: # Stop condition
18             break
19     return Lambda, J_hist, Grad_hist

```

Convexit  de la fonction c  t et unicit  du minimum global

La fonction c  t J est une fonction quadratique dont le **Hessien** est donn  par:

$$\nabla^2 J(\theta) = \frac{1}{n} {}^tXX$$

Comme tXX est **semi-d finie positive**, J est **convexe**.

Ainsi, **tout minimum local est global**.

De plus, si tXX est **inversible**, alors J est **strictement convexe** et admet un **unique minimum global**.